

PRODUCTION-SITE-HEALTH

Production site health — verification report

Verified at: 2026-08-27T19:52:13Z (UTC) **Verified from:** repository root
<ext-volume-host>/Projects/vasic **Scope:** every website this repository
deploys. **Mode:** read-only. No site content, workflow, or config was
modified. Nothing was committed or pushed.

Every claim below is backed by a command whose exact text is
reproduced so the check can be re-run. Where a check could not be
performed, the item is marked UNVERIFIED with the reason (§11.4.3 /
§11.4.6). A 200 status code is never treated on its own as proof of
correct publishing.

Executive summary

Metric	Value
Websites deployed by this repository	2
HEALTHY	2
AT RISK	0
BROKEN	0
Live URLs checked	1050 (full sitemap sweep, both sites)
Live URLs returning 200	1050 / 1050
Pages rendered correctly (sampled, deep-checked)	88 / 88

Is any website currently broken? No. Is any website currently at risk of breaking? No — for the live sites.

Both production websites — <https://milosvasic.ru/> and <https://vasic.digital/> — serve HTTP 200 over valid TLS, publish through an ACTIVE, green publishing mechanism, and serve genuinely rendered output (zero raw Liquid, zero leaked YAML front matter across an 88-page deep sample and a 1050-URL sitemap sweep). The most recent deployment of each site succeeded and corresponds to the exact commit that is currently `origin/main HEAD` in each site repository.

Items needing operator action

None are site-breaking. Three items are recorded for the operator's decision:

#	Severity	Item
A1	Advisory (deploy tooling, not the live sites)	<code>_tests/node_modules</code> is not installed on this machine, so the post-deploy LIVE validation step inside <code>_tools/deploy-langs.sh</code> (lines 123-124, <code>npx playwright test</code>) cannot run cleanly here. <code>deploy-langs.sh</code> treats a validator failure as <code>LIVE_FAIL=1</code> and exits non-zero, so a deploy run on this machine would report LIVE validation: FAIL — broken links on live for a tooling reason rather than a real broken link. Fix before the next deploy: <code>cd _tests && npm ci && npx playwright install chromium</code> . The sites themselves are unaffected — this report's independent

#	Severity	Item
A2	Content / SEO defect on milosvasic.ru (not an outage)	1050-URL sweep already covers what that validator checks. All 14 localized homepages and all localized product pages serve the English <code><title></code> and <code><meta name="description"></code>. The generated front matter for <code><lang>/index.html</code> omits <code>title:</code> and <code>description:</code>, so the Jekyll layout falls back to the site default. <code>vasic.digital</code> does not have this defect — its localized titles are correctly translated. Separately, <code>milosvasic.ru</code> product-page titles duplicate the site name: <pre><title>HelixTerminator – Miloš Vasić \ Miloš Vasić</title>.</pre> <code>_tools/deploy-langs.sh</code> runs <code>git -C "\$s" add -A</code> (line 91) before committing each site. At the time of this report <code>milosvasic.ru</code> has a staged, uncommitted
A3	Coupling risk in the deploy script	Upstreamable gitlink change (94f9831 → 9bf1240) placed there by a concurrent agent. A deploy run right now would sweep that gitlink change into a production content commit. This does not

#	Severity	Item
		break the build (pages.yml sets submodules: false and _config.yml excludes Upstreamable from the Jekyll build), but it couples an unrelated in- progress repair to a production publish.

1. Site discovery — how the list of 2 was established

The list was not assumed. Six independent discovery paths were run and all converge on exactly two deployed websites.

1. Declared submodules

```
cat .gitmodules
git submodule status
```

2. Custom-domain markers

```
find . -name CNAME -not -path '*/node_modules/*' -not -path '*/_site/*'
```

3. Jekyll sources

```
find . -maxdepth 3 -name '_config.yml' -not -path '*/node_modules/*'
```

4. Tracked workflows, root + every submodule

```
git ls-files .github
for d in milosvasic.ru vasic.digital design-toolkit
    ai_interviewing monetization; do
    (cd "$d" && git ls-files .github)
done
```

5. Deploy scripts

```
ls -la _tools/ # deploy-langs.sh is the only site
               deploy driver
sed -n '1,200p' _tools/deploy-langs.sh
```

6. GitHub Pages configuration for every candidate repo

```
for r in milos85vasic/milosvasic.ru vasic-digital/vasic-
    digital.github.io \
    vasic-digital/design-toolkit milos85vasic/
    ai_interviewing \
    milos85vasic/monetization milos85vasic/vasic; do
```

```
gh api "repos/$r/pages"
gh api "repos/$r" --jq '"has_pages=(.has_pages) homepage=(.homepage)"'
```

done

Results

These `build_type` values are a dated observation, not a standing fact. Provider settings change outside this tree and nothing in it can see that happen. Re-measure with `bash scripts/verify-provider-ci.sh` — it discovers the repositories from this checkout's own remotes and queries the provider (exit 0 none found, 1 confirmed provider-side triggering, 2 could not determine, which is **not** a pass). `scripts/setup-agents-wizard.sh` runs it as Step 9.

Repository / path	CNAME	Jekyll	Tracked workflow	GitHub Pages
milosvasic.ru/ → milos85vasic/ milosvasic.ru	milosvasic.ru	yes (<code>_config.yml</code>)	.github/workflows/pages.yml	has_pages=true, build_type=workf
vasic.digital/ → vasic-digital/ vasic-digital.github.io	vasic.digital	no (committed static HTML)	none (uses GitHub's built-in)	has_pages=true, build_type=legac
design-toolkit/ → vasic-digital/ design-toolkit	none	none	none	has_pages=false (API 404)
ai_interviewing/ → milos85vasic/ ai_interviewing	none	none	none	has_pages=false (API 404)
monetization/ → milos85vasic/ monetization	none	none	none	has_pages=false (API 404)
submodules/ constitution, submodules/ superspec	none	none	n/a	not queried as si (no CNAME, no Pages markers)
umbrella repo milos85vasic/ vasic	none	none	.github/workflows/ci.yml.disabled (see §5.4)	has_pages=false (API 404)

A domain sweep over README.md, docs/, and _tools/*.sh surfaced no third site domain — only milosvasic.ru plus third-party/tooling hosts:

```
grep -rhoE 'https?://[a-zA-Z0-9._-]+\.[a-z]{2,}' README.md docs/_tools/*.sh \
| sed 's|https\?://||' | sort -u
```

_tools/deploy-langs.sh itself iterates exactly for s in vasic.digital milosvasic.ru (lines 74 and 90), confirming two deploy targets from the deploy driver’s own code.

Tooling availability: gh version 2.86.0, authenticated as milos85vasic, token scopes gist, read:org, repo, workflow. All gh api results below are real API responses, not inferred.

2. Site 1 — https://milosvasic.ru/

Verdict: HEALTHY

Jekyll site. Source in submodule milosvasic.ru/. _site/ is git-ignored and built by GitHub Actions.

Evidence

Check	Result	Evidence
Live HTTP status	200	200 https://milosvasic.ru/
www. host	200, redirects to apex	200 https://milosvasic.ru/ from https://www.milosvasic.ru
http:// → https://	200, lands on https://milosvasic.ru/	https_enforced: true in Pages API
TLS valid	YES — ssl_verify_result=0	Let’s Encrypt CN=YR1, CN=milosvasic.ru, SAN milosvasic.ru, www.milosvasic.ru, notAfter=Nov 20 06:08:24 2026 GMT
last-modified	Thu, 27 Aug 2026 00:12:48 GMT	curl -sSI

Check	Result	Evidence
Server	GitHub.com, edge region fra	response headers
DNS	185.199.108.153 .109.153 .110.153 .111.153 (GitHub Pages apex IPs), apex and www identical	getent ahostsv4
Publish mechanism	GitHub Pages, build_type: workflow	gh api repos/ milos85vasic/ milosvasic.ru/pages → "build_type":"workflow", "status":"built", "cname":"milosvasic.ru", "https_enforced":true
Publishing workflow tracked and ACTIVE	YES	git ls-files .github → .github/workflows/ pages.yml; gh api .../ actions/workflows → Deploy Jekyll to GitHub Pages \ state=active \ path=.github/ workflows/pages.yml
Workflow committed, no local drift	YES	git status --short clean for that path; git diff HEAD -- .github/ workflows/pages.yml empty; introduced by fae3b24, confirmed ancestor of origin/main
Last 8 Actions runs	8 / 8 success	latest: Deploy Jekyll to GitHub Pages \ completed \ success \ 2026-08-27T00:11:55Z \ 66c8d60
Last run, job level	build success, deploy success, 0 failed steps	run 33026023067
Last Pages deployment	success at 2026-08-27T00:12:54Z, env URL https://milosvasic.ru/, sha 66c8d60	deployments API
Deployed sha == origin/main	YES — 66c8d607b20f0d9e984cb0e06f10a2020ebd9a10 both sides; 0 0 ahead/behind	git rev-list --left- right --count HEAD...origin/main
Rendered output, not	YES	homepage: {{ count 0, {% count 0, no leading ---,

Check	Result	Evidence
template source		<title> present, </html> present
Full sitemap sweep	525 / 525 URLs → 200	see §3
Deep page sample	44 pages, 0 with raw Liquid, 0 with front matter	see §3
Branch protection on main	NONE — API returns 404 Branch not protected (permission was sufficient; token has repo scope and other protected endpoints answered)	gh api repos/milos85vasic/milosvasic.ru/branches/main/protection
github-pages environment	protection_rules: [branch_policy], custom_branch_policies: true — no reviewer gate that could stall a deploy	environments API
Repo state	default_branch=main private=false archived=false disabled=false has_pages=true	repo API

Jekyll is genuinely rendering — positive proof

The strongest single proof that the build is live and not frozen: the **committed source has YAML front matter and the served page does not**. If Jekyll were not running, the front matter would leak verbatim.

```
git -C milosvasic.ru show HEAD:de/index.html | head -13
# begins with ---\nlayout: default\nlang: de ...
curl -sS -L https://milosvasic.ru/de/ | head -3
# begins with <!DOCTYPE html>...
```

- Committed de/index.html: 33 343 bytes, starts --- / layout: default / lang: de.
- Served /de/: 40 352 bytes, starts <!DOCTYPE html>, <html lang="de" dir="ltr" data-theme="light">.
- The committed <h1 class="od-hero__title">Miloš Vasić is **present** in the served page.
- Served /de/ contains **16** rel="alternate" hreflang links — the seo_hreflang front-matter value was consumed and expanded by the layout.
- Served /ar/ correctly carries dir="rtl".

Reproduce

```
curl -sS -o /dev/null -m 25 -w '%{http_code} %{url_effective}
      tls_verify=%{ssl_verify_result}\n' -L https://
      milosvasic.ru
curl -sSI -m 25 -L https://milosvasic.ru/ | grep -iE '^(HTTP/|
      last-modified|server)'
echo | openssl s_client -servername milosvasic.ru -connect
      milosvasic.ru:443 2>/dev/null \
      | openssl x509 -noout -subject -issuer -dates -ext
      subjectAltName
gh api repos/milos85vasic/milosvasic.ru/pages
gh api repos/milos85vasic/milosvasic.ru/actions/workflows \
      --jq '.workflows[] | "\(.name) | state=\(.state) | path=\
      (.path)"'
gh api 'repos/milos85vasic/milosvasic.ru/actions/runs?per_page=8'
      \
      --jq
      '.workflow_runs[] | "\(.name) | \(.status) | \
      (.conclusion) | \(.created_at) | \(.head_sha[0:7])"'
gh api repos/milos85vasic/milosvasic.ru/actions/runs/33026023067/
      jobs \
      --jq '.jobs[] | "\(.name) | \(.status) | \(.conclusion)'"
gh api 'repos/milos85vasic/milosvasic.ru/deployments?
      environment=github-pages&per_page=3' \
      --jq '.[0] | "\(.created_at) sha=\(.sha[0:7])"'
gh api repos/milos85vasic/milosvasic.ru/branches/main/protection
gh api repos/milos85vasic/milosvasic.ru/environments/github-pages
      \
      --jq '{protection_rules:[.protection_rules[]?|{type:.type}],
      deployment_branch_policy:.deployment_branch_policy}'
git -C milosvasic.ru fetch origin main && git -C milosvasic.ru
      rev-list --left-right --count HEAD...origin/main
```

Defects recorded against this site (non-breaking)

See A2 in the executive summary. Evidence:

```
for l in "" de/ ja/ ru/ ar/; do
  curl -sS -L "https://milosvasic.ru/$l" | grep -o '<title>[^<]*</
    title>' | head -1
done
```

All five return the identical English string <title>Miloš Vasić | AI Engineer – Verifiable AI-Development Systems</title>, while <html lang=...> is correctly localized (en, de, ja, ru, ar). Root cause, verified in the committed source:

```
for l in ru sr de es fr be zh kk hi ja ko ar tr fa; do
  git -C milosvasic.ru show "HEAD:$l/index.html" | sed -n '1,15p'
  | grep -qE '^title:' \
  && echo "$l has title" || echo "$l NO title in front matter"
```

done

```
# → all 14 report "NO title in front matter"
```

Product pages additionally double the site name:

<title>HelixTerminator – Miloš Vasić | Miloš Vasić</title> — the front-matter title already ends in – Miloš Vasić and the layout appends | Miloš Vasić.

3. Content sanity — full sweep across both sites

A 200 alone proves nothing, so the content itself was fetched and inspected.

3.1 Full sitemap status sweep — 1050 URLs

```
for h in milosvasic.ru vasic.digital; do
  curl -sS -L "https://$h/sitemap.xml" -o "sm-$h.xml"
  grep -o '<loc>[^<]*</loc>' "sm-$h.xml" | sed 's/<[^>]*>//g' >
    "locs-$h.txt"
done
cat locs-*.txt > locs-all.txt # 1050 URLs (525 per site)
cat locs-all.txt | xargs -P 10 -I{} \
  curl -sS -o /dev/null -m 30 -L -w '%{http_code} {} \n' {} >
  sitemap-status.txt
awk '{print $1}' sitemap-status.txt | sort | uniq -c | sort -rn
grep -v '^200 ' sitemap-status.txt
```

Result: 1047 × 200, 3 × 503. All three 503s were on milosvasic.ru and were **transient GitHub Pages rate-limiting from the parallel sweep** — each returned 200 on immediate serial retry:

```
retry: 200 https://milosvasic.ru/products/helixterminator.html
retry: 200 https://milosvasic.ru/products/kk/helixllm.html
retry: 200 https://milosvasic.ru/products/ko/helixbuilder.html
```

Effective result: 1050 / 1050 → 200.

3.2 Deep render check — 88-page spread sample

Every 12th URL of the combined sitemap (88 pages, both sites, all 15 languages, homepages + language indexes + portfolio + product pages) was downloaded in full and inspected for build breakage:

```

awk 'NR%12==1' locs-all.txt > sample.txt          # 88 URLs
i=0; while read u; do i=$((i+1)); curl -sS -m 30 -L "$u" -o
    "pages/p$i.html"; done < sample.txt

grep -l '{{' pages/*.html | wc
    -l                                           # → 0
grep -l '{%' pages/*.html | wc
    -l                                           # → 0
for f in pages/*.html; do head -c 4 "$f" | grep -q '^---' && echo
    "$f"; done | wc -l    # → 0
for f in pages/*.html; do grep -q '</html>' "$f" || echo "MISSING
    </html>: $f"; done    # → none
for f in pages/*.html; do grep -q '<title>' "$f" || echo
    "MISSING <title>: $f"; done    # → none
ls -S pages/*.html | tail -5 | xargs wc -c    # smallest page is
    29 226 bytes – no stub/empty output

```

Signal	Threshold for BROKEN	Observed
Pages containing raw Liquid {{	any	0
Pages containing raw Liquid {%	any	0
Pages starting with YAML front matter ---	any	0
Pages missing </html>	any	0
Pages missing <title>	any	0
Smallest rendered page	stub-sized	29 226 bytes

3.3 Ancillary endpoints

```

for h in milosvasic.ru vasic.digital; do
    for p in robots.txt sitemap.xml feed.xml 404.html
        _definitely_missing_; do
        curl -sS -o /dev/null -m 20 -w "%{http_code} $h/$p\n" -L
            "https://$h/$p"
    done
done

```

Endpoint	milosvasic.ru	vasic.digital
robots.txt	200	200
sitemap.xml	200 (525 <loc>)	200 (525 <loc>)
feed.xml	200 (Jekyll feed)	404 — expected , static site with no Jekyll feed generator
unknown path	404	404

4. Site 2 — <https://vasic.digital/>

Verdict: HEALTHY

Committed static HTML, served as-is by GitHub Pages' built-in pages-build-deployment. No custom workflow exists or is needed.

Evidence

Check	Result	Evidence
Live HTTP status	200	200 https://vasic.digital/
www. host	200 , redirects to apex	200 https://vasic.digital/ from https://www.vasic.digital
vasic-digital.github.io	200 , redirects to https://vasic.digital/	custom domain correctly bound
http:// → https://	200 , lands on https://vasic.digital/	https_enforced: true
TLS valid	YES — ssl_verify_result=0	Let's Encrypt CN=YR1, CN=vasic.digital, SAN vasic.digital, www.vasic.digital, notAfter=Oct 8 16:32:52 2026 GMT
last-modified	Thu, 27 Aug 2026 00:15:26 GMT	curl -sSI
Server	GitHub.com, edge region fra	response headers
DNS	185.199.108.153 .109.153 .110.153 .111.153, apex and www identical	getent ahostsv4
Publish mechanism	GitHub Pages, build_type: legacy (built-in builder, source.branch=main, path=/)	gh api repos/vasic-digital/vasic-digital.github.io/pages → "build_type": "legacy", "status": "built", "cname": "vasic.digital"
Tracked publish workflow in repo	NONE — and correctly so	git ls-files .github vasic.digital/ return nothing; publishing is

Check	Result	Evidence
Last 8 Actions runs	8 / 8 success	GitHub's built-in dynamic/pages/pages-build-deployment, reported state=active latest: pages build and deployment \ complete \ success \ 2026-08-27T00:14:51Z 6e5411c
Last run, job level	build success, deploy success, report-build-status success	run 33026192751
Last 5 Pages builds	5 / 5 built, error=null	gh api .../pages/builds
Last Pages deployment	success at 2026-08-27T00:15:32Z, env URL https://vasic.digital/, sha 6e5411c	deployments API
Deployed sha == origin/main	YES — 6e5411c21b3cd9c6df5addb543b1a07930c3bfa9 both sides; 0 0 ahead/behind	git rev-list --left-right --count HEAD...origin/main
Working tree clean	YES	git -C vasic.digital status --short empty
Rendered output, not template source	YES	{{ count 0, {% count 0 no leading ---, <title>Vasic Digital AI-Native Software Engineering</title>, html> present
Live bytes == committed bytes	BYTE-IDENTICAL (48 456 bytes)	see below — decisive proof the site is not frozen
Full sitemap sweep	525 / 525 URLs → 200	see §3
Localized titles	correctly translated in de / ja / ru / ar	see below
Branch protection on main	NONE — API returns 404 Branch not protected	gh api repos/vasic-digital/vasic-digital.github.io/branches/main/protection
github-pages environment		environments API

Check	Result	Evidence
Repo state	protection_rules: [branch_policy], custom_branch_policies: true — no reviewer gate default_branch=main private=false archived=false disabled=false has_pages=true	repo API

Decisive freshness proof — live equals committed HEAD, byte for byte

Because this site is committed static HTML, the served page can be diffed directly against the commit that is origin/main:

```
git -C vasic.digital show HEAD:index.html > vd-committed.html
curl -sS -L https://vasic.digital/ -o vd-live.html
diff -q vd-committed.html vd-live.html
```

Result: **identical**, 48 456 bytes on both sides. The live site is serving exactly the content of commit 6e5411c, which is exactly origin/main. This rules out a frozen or stale deployment.

Localized rendering

```
for l in "" de/ ja/ ru/ ar/; do
  curl -sS -L "https://vasic.digital/$l" | grep -o '<title>[^<]*</title>' | head -1
done
```

```
<title>Vasic Digital – AI-Native Software Engineering</title>
<title>Vasic Digital – KI-natives Software Engineering</title>
<title>Vasic Digital – AIネイティブ ソフトウェアエンジニアリング</title>
<title>Vasic Digital – AI-нативная разработка программного
обеспечения</title>
<title>Vasic Digital – هندسة برمجيات AI-Native</title>
```

<html lang> values are en / de / ja / ru / ar, with dir="rtl" on Arabic.

Reproduce

```
curl -sS -o /dev/null -m 25 -w '%{http_code} %{url_effective}
  tls_verify=%{ssl_verify_result}\n' -L https://
  vasic.digital
curl -sSI -m 25 -L https://vasic.digital/ | grep -iE '^(HTTP/|
  last-modified|server)'
```

```

echo | openssl s_client -servername vasic.digital -connect
      vasic.digital:443 2>/dev/null \
    | openssl x509 -noout -subject -issuer -dates -ext
      subjectAltName
gh api repos/vasic-digital/vasic-digital.github.io/pages
gh api repos/vasic-digital/vasic-digital.github.io/actions/
      workflows \
    --jq '.workflows[] | "\(.name) | state=\(.state) | path=\
      (.path)"'
gh api 'repos/vasic-digital/vasic-digital.github.io/actions/runs?
      per_page=8' \
    --jq
      '.workflow_runs[] | "\(.name) | \(.status) | \
      (.conclusion) | \(.created_at) | \(.head_sha[0:7])"'
gh api 'repos/vasic-digital/vasic-digital.github.io/pages/builds?
      per_page=5' \
    --jq '.[0] | "\(.status) | \(.created_at) | \(.commit[0:7]) |
      err=\(.error.message) "'
gh api repos/vasic-digital/vasic-digital.github.io/branches/main/
      protection
git -C vasic.digital fetch origin main && git -C vasic.digital
      rev-list --left-right --count HEAD...origin/main

```

5. Risk review

5.1 milosvasic.ru/.github/workflows/pages.yml — present, ACTIVE, intact

This is the **sole publish path** for milosvasic.ru (the repo's Pages build_type is workflow, so GitHub's built-in Jekyll builder is bypassed). It was verified on all five required points:

Requirement	Status	Evidence
Present on disk	YES	milosvasic.ru/.github/workflows/pages.yml, 1333 bytes
Tracked by git	YES	git ls-files .github → .github/workflows/pages.yml
Committed with no local drift	YES	git diff HEAD -- .github/workflows/pages.yml empty; git status --short shows no entry for it
	YES	

Requirement	Status	Evidence
Present on origin/main		git merge-base --is-ancestor fae3b24 origin/main → ancestor confirmed
Registered ACTIVE on GitHub	YES	gh api .../actions/workflows → state=active
Parses as valid YAML	YES	yaml.safe_load succeeded
build job intact	YES	5 steps: actions/checkout@v4 (submodules: false), actions/configure-pages@v5, ruby/setup-ruby@v1 (3.3, bundler-cache), bundle exec jekyll build --destination _site (JEKYLL_ENV=production), actions/upload-pages-artifact@v3
deploy job intact	YES	needs: build, environment: github-pages, 1 step actions/deploy-pages@v4 with id: deployment
Triggers intact	YES	on: {push: {branches: [main]}}, workflow_dispatch: null}
Permissions intact	YES	contents: read, pages: write, id-token: write
Last execution	success, both jobs, 0 failed steps, 2026-08-27T00:11:55Z on 66c8d60	run 33026023067

```
python3 -c "
import yaml
d = yaml.safe_load(open('milosvasic.ru/.github/workflows/pages.yml'))
print('jobs:', list(d['jobs'].keys()))
```



```

print('build steps:', len(d['jobs']['build']['steps']))
print('deploy steps:', len(d['jobs']['deploy']['steps']))
print('deploy needs:', d['jobs']['deploy']['needs'])
"
# jobs: ['build', 'deploy'] / build steps: 5 / deploy steps: 1 /
    deploy needs: build

```

5.2 Why submodules: false matters — do not remove it

`pages.yml` exists specifically because the default recursive checkout fails on a broken nested gitlink in `milosvasic.ru/Upstreamable`. That failure is not hypothetical — the legacy builder errored **four times** before the workflow was introduced:

```

gh api repos/milos85vasic/milosvasic.ru/pages/builds \
  --jq '.[ ] | select(.status=="errored") | {created:.created_at,
    commit:.commit[0:7], error:.error.message}'

```

```

{"commit":"f120ac6","created":"2026-08-06T17:04:48Z","error":"Page
build failed."}
{"commit":"fd2f66e","created":"2026-08-06T14:46:39Z","error":"Page
build failed."}
{"commit":"d68aa97","created":"2026-08-06T11:22:23Z","error":"Page
build failed."}
{"commit":"72a29d5","created":"2026-08-06T10:47:33Z","error":"Page
build failed."}

```

GitHub's API returns only the generic string `Page build failed.` — the underlying runner log is not exposed through this endpoint, so the specific error text is **UNVERIFIED** beyond that message. The chronology is unambiguous, however: the errors stop the moment `pages.yml` lands, and every run since is green.

Operator guidance: `submodules: false` in `pages.yml` must not be flipped to `true` and `pages.yml` must not be deleted (which would fall back to the legacy builder that fails). `Upstreamable` is excluded from the Jekyll build anyway — `milosvasic.ru/_config.yml` line 45:

```

exclude:
  - Gemfile
  - Gemfile.lock
  - README.md
  - README.pdf
  - LICENSE
  - start.sh
  - vendor
  - Upstreamable      # ← line 45
  - Upstreams

```

- .run
- downloads/src

Minor doc drift: the comment at the top of `pages.yml` cites “`_config.yml` line 43”; the exclusion is now at line 45. Harmless — the exclusion itself is intact.

5.3 Concurrent Upstreamable work — currently harmless, but coupled

At report time `milosvasic.ru` carries one **staged, uncommitted** change, placed by a concurrent agent (not touched by this verification):

```
git -C milosvasic.ru status --short
# M Upstreamable
git -C milosvasic.ru diff --cached -- Upstreamable
# -Subproject commit 94f9831b8aa0a1d4df23671d2e4600886aad0dcf
# +Subproject commit 9bf124096eda44e78edcd3f56df748d23ef4c577
```

Impact on publishing: **none today**. `pages.yml` checks out with `submodules: false` and `_config.yml` excludes `Upstreamable`, so the gitlink value is irrelevant to the build. The residual risk is procedural and is recorded as **A3**: `deploy-langs.sh` line 91 runs `git -C "$s" add -A`, so the next deploy would fold this in-progress gitlink change into a production content commit.

5.4 Root `ci.yml` → `ci.yml.disabled` — recent change, not a publish path

The umbrella repository’s CI workflow was renamed and the rename is **staged but not yet committed**:

```
git ls-files .github          # → .github/workflows/
    ci.yml.disabled
git status --short .github    # → R .github/workflows/ci.yml
    -> .github/workflows/ci.yml.disabled
```

Verified that this workflow never published a site:

```
python3 -c "
import yaml
d = yaml.safe_load(open('.github/workflows/ci.yml.disabled'))
print('name:', d.get('name')); print('jobs:',
    list(d['jobs'].keys()))
"
# name: CI / jobs: ['quality-gates']
```

```
grep -niE 'actions/deploy-pages|upload-pages-artifact|gh-
      pages' .github/workflows/ci.yml.disabled
# → no matches in any executable step (only prose in header
      comments)
```

It contained a single quality-gates job and no deploy step, and the umbrella repo has `has_pages=false`. Its own header comment states the enforcement moved to a local pre-push hook (`scripts/pre-push-gates.sh`). **No publishing impact.** This also closes gap G4 recorded in CLAUDE.md (root `ci.yml` was flagged as conflicting with anchor 11.4.156(A)); note the rename is not yet committed, so the resolution is not yet durable.

5.5 Branch protection and required status checks

```
gh api repos/milos85vasic/milosvasic.ru/branches/main/protection
gh api repos/vasic-digital/vasic-digital.github.io/branches/main/
  protection
```

Both return:

```
{"message":"Branch not protected","status":"404"}
```

Read as: **no branch protection and no required status checks on main in either site repo.** The 404 is the documented “not protected” response, not a permission denial — the token carries repo scope and the sibling protected endpoints (`/environments/github-pages`, `/deployments`) answered successfully for the same repos. Nothing can block a merge or a push, so nothing can stall a publish.

The `github-pages` environment in both repos carries only a `branch_policy` rule with `custom_branch_policies: true` and `protected_branches: false` — no required reviewer, so no deployment can sit waiting for approval.

5.6 Recent workflow errors

```
gh api 'repos/milos85vasic/milosvasic.ru/actions/runs?per_page=8'
  \
  --jq '.workflow_runs[] | "\(.name) | \(.status) | \
      (.conclusion) | \(.created_at)'"
gh api 'repos/vasic-digital/vasic-digital.github.io/actions/runs?
  per_page=8' \
  --jq '.workflow_runs[] | "\(.name) | \(.status) | \
      (.conclusion) | \(.created_at)'"
```

Zero Actions failures on either site repo across the last 8 runs each (16 runs, 16 × success), back to 2026-08-07. The only errors anywhere in the publishing history are the four legacy Pages builds of 2026-08-06 documented in §5.2, all of which predate and motivated the current workflow.

5.7 Deploy tooling readiness

_tools/deploy-langs.sh ends with a post-deploy LIVE validation gate:

```
sed -n '118,127p' _tools/deploy-langs.sh
# ( cd "$ROOT/_tests" && VD_BASE="https://$VD_DOMAIN"
#     MV_BASE="https://$MV_DOMAIN" \
#     npx playwright test --config=playwright.live.config.js all-
#         languages-link-integrity.spec.js --reporter=line )
```

Verified wiring:

Item	Status
Spec file exists and is tracked	YES — _tests/tests/all-languages-link-integrity.spec.js
Config resolves it	YES — playwright.live.config.js has testDir: './tests' and testMatch includes all-languages-link-integrity\spec\js
Env-var contract matches	YES — spec reads process.env.VD_BASE / process.env.MV_BASE; the script exports exactly those two
_tests/node_modules installed	NO — ls: cannot access '_tests/node_modules'

The missing node_modules is item **A1**: deploy-langs.sh maps any validator non-zero exit to LIVE_FAIL=1 and exits 1 with the message LIVE validation: FAIL – broken links on live, which on this machine would be a tooling artefact, not a real broken link. Remedy before the next deploy:

```
cd _tests && npm ci && npx playwright install chromium
```

This report’s §3 sitemap sweep and 88-page render sample were run independently of that harness and already establish live link integrity for both sites.

6. Explicitly UNVERIFIED

Recorded honestly per §11.4.6 — these were **not** proven and must not be read as passes:

Item	Why unverified
Specific error text of the four 2026-08-06 legacy Pages build failures	gh api .../pages/builds exposes only the generic string Page build failed.; the legacy builder’s runner log is not retrievable through the API. The correlation with the Upstreamable gitlink is stated in pages.yml’s own header comment and is consistent with the chronology, but the raw error text was not read.
A local end-to-end Jekyll rebuild of milosvasic.ru	Not run — it would mutate the working tree and could collide with the concurrent Upstreamable work. Build health is instead evidenced by the green GitHub Actions run on the exact current HEAD plus the front-matter-consumed / hreflang-expanded rendering proof in §2.
The Playwright live-validation suite itself	Not executed — _tests/node_modules is absent (item A1). An equivalent independent check (1050-URL sweep + 88-page render inspection) was run instead and is reported in §3.
Whether GitHub Pages usage quotas or org-level Actions policies could throttle a future deploy	Not queried. Out of scope.

Item	Why unverified
Health of any site outside this repository	

7. Bottom line

- **2 websites are deployed by this repository.** Both were found by discovery, not assumed.
- **<https://milosvasic.ru/> — HEALTHY.** 200, valid TLS to 2026-11-20, publishing via ACTIVE workflow `pages.yml` (`build_type: workflow`), last deploy success on the exact current origin/main sha, 525/525 sitemap URLs live, fully rendered output. Two non-breaking SEO defects recorded (A2).
- **<https://vasic.digital/> — HEALTHY.** 200, valid TLS to 2026-10-08, publishing via GitHub's built-in builder (`build_type: legacy`), last deploy success on the exact current origin/main sha, 525/525 sitemap URLs live, and the served homepage is **byte-identical** to the committed source.
- **No website is broken. No website is at risk of going down.**
- The only pre-deploy chore is **A1** (`cd _tests && npm ci && npx playwright install chromium`), which affects the deploy script's self-check, not the live sites.